... there might be a `Content-Type:` header, perhaps others too, but it's still far simpler than a `multipart/form-data` request. Let's see how PHP behaves; create the following `index.php`:

```
// index.php

var_dump($_FILES);
var_dump($_POST);
var_dump(file_get_contents('php://input'));
```

Adding the `--upload-file` option to a curl command will trigger a PUT request without a `Content-Type:` header. Create a file `meow.txt` in your working directory and add a line of text to it, ideally something feline in nature. Now run the following curl command:

```
$ curl -v --upload-file meow.txt 'http://localhost:8080/index.php'
> PUT /index.php HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/8.7.1
> Accept: */*
> Content-Length: 47
...
array(0) {
}
array(0) {
}
string(47) "Jess is pretending she hasn't been fed. Again.
"
* Closing connection
```

Here we can see that both `$_FILES` and `$_POST` are empty. PHP takes no special action with the request data of a PUT. We have to go to STDIN (via the `file_get_contents('php://input')` call) to access the request body. You can imagine the rest for yourself: read from `php://input`, write to a file handle obtained from `fopen()`, and so on.

It is perfectly fine (from HTTP's perspective) to make a `multipart/form-data` PUT request; we can get curl to do this by using a `-F` option to trigger the desired `Content-Type:` and `-X 'PUT'` to override the default HTTP method: